



Factory Method: Patron de diseño Creacional.

Paola Ramírez Salazar | *Departamento de Ingeniería.*
Yucsibet Azocar Gonzalez.

Situacion...

- El desarrollo de un Sistema de gestion logistica.

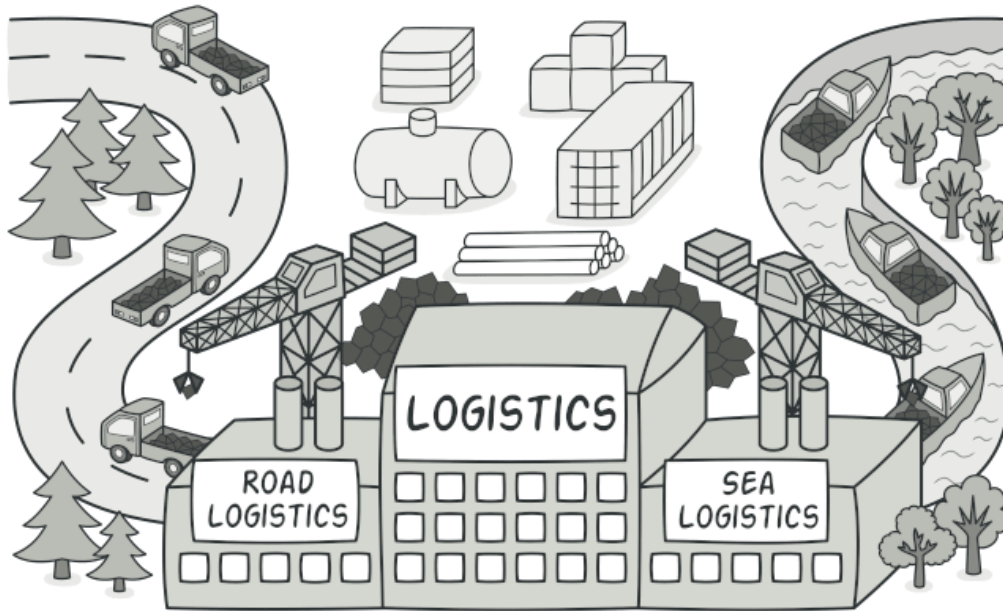


Fig 1.

- Instancia objetos de la clase camion con new.



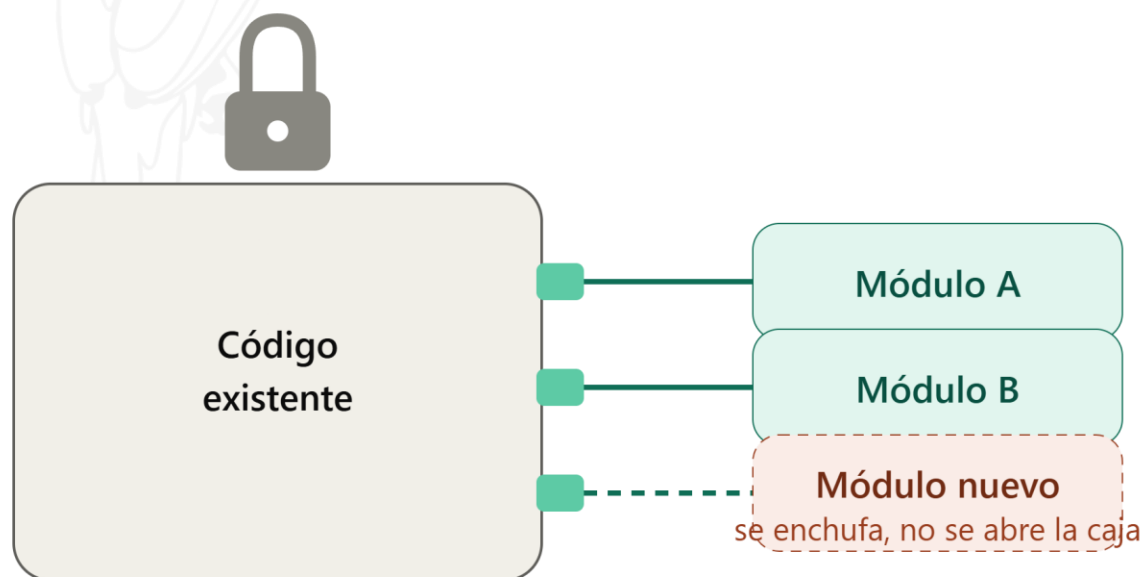
Fig 2.

○ El problema surge...

- Para incorporar logistica por barco y camion.

El problema rompe con dos principios fundamentales...

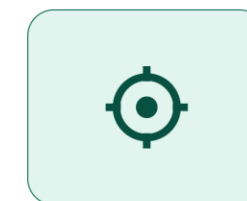
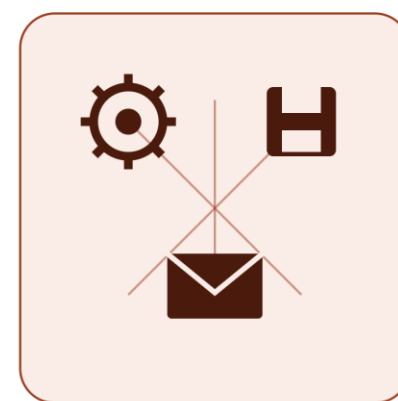
1. Principio Abierto/Cerrado (Open/Closed Principle).



cerrada: nadie la abre ni la modifica

abierta: se agregan módulos por los puertos

2. Principio de Responsabilidad Única (Single Responsibility Principle).



calcular



guardar



enviar
tres clases, una responsabilidad cada una

una clase, tres responsabilidades enredadas

Fig 3.

A la Verdad. por la fe y la ciencia

La Solución :Factory Method.

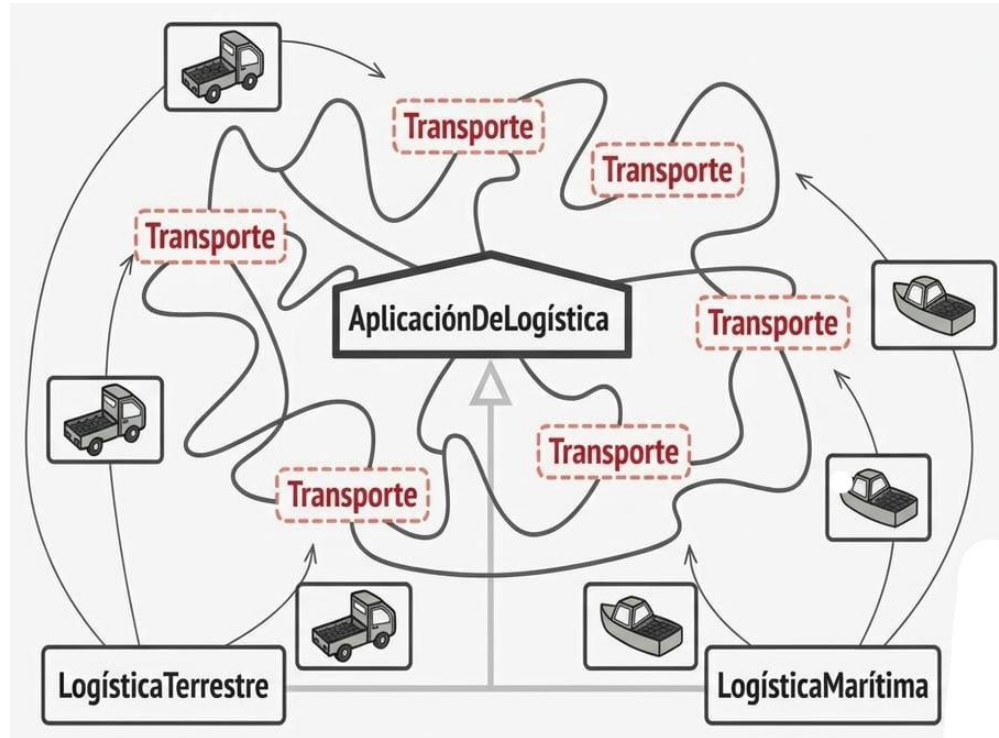


Fig 3.

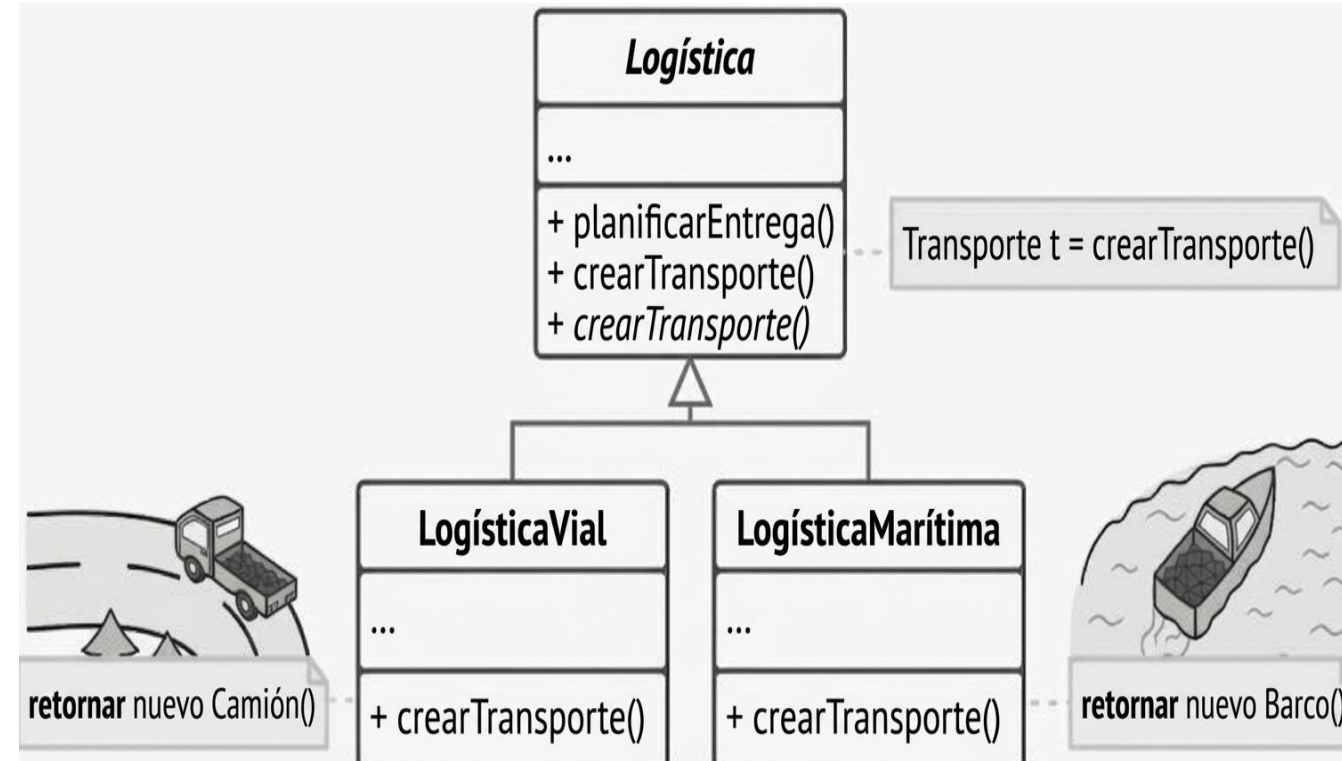


Fig 4.

La Estructura UML

1. Producto (Transporte).
2. Productos Concretos (Camión, Barco).
3. Creador (Logística).
4. Creadores Concretos (Logística Terrestre, Logística Marítima)

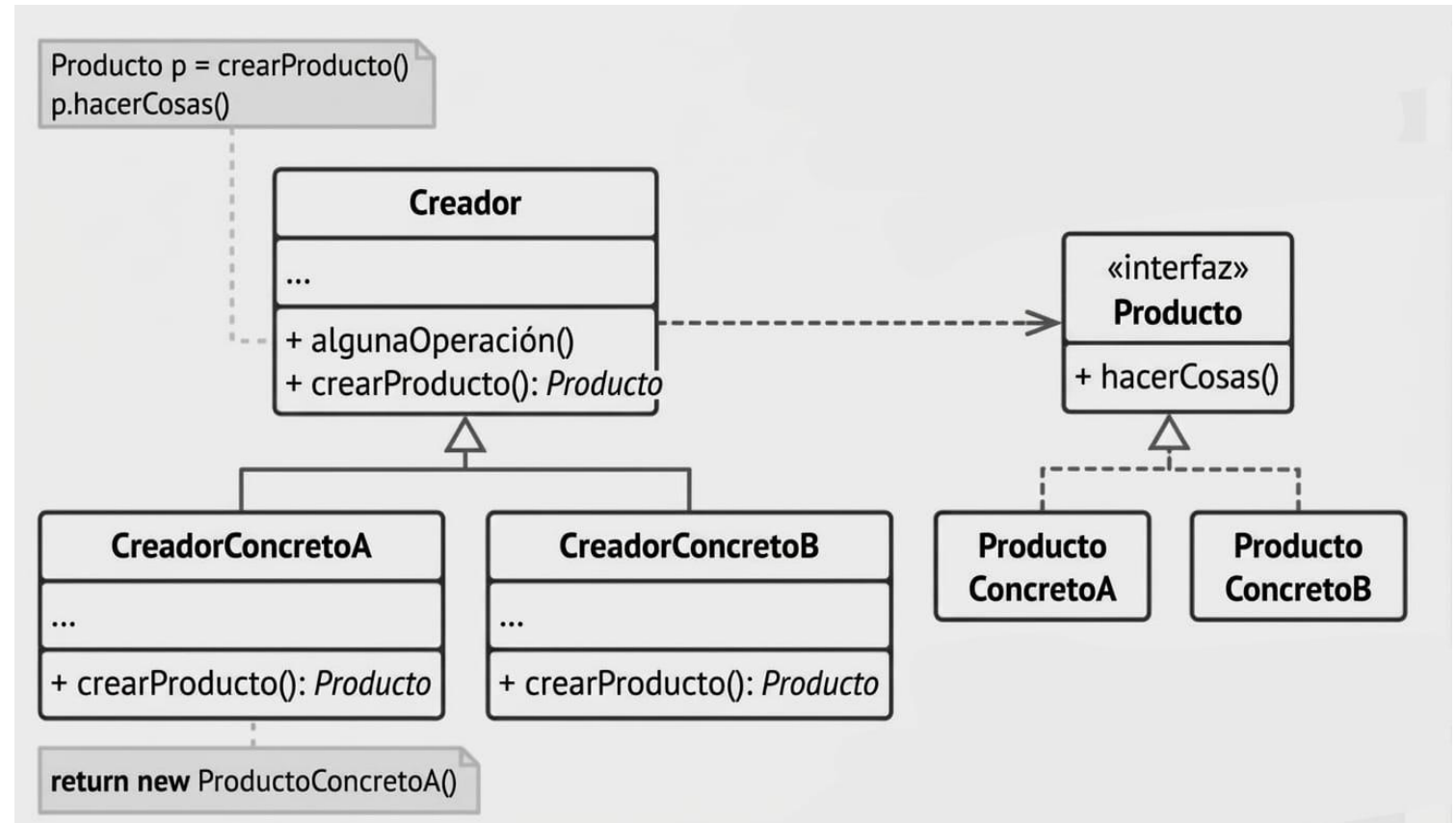


Fig 5.

Ejemplo

Supongamos que estamos creando un juego que tiene diferentes tipos de personajes, como guerreros, magos y arquero. Cada uno de estos personajes tiene un conjunto específico de habilidades y ataques.

- Sin Factory Method:

```
public class Guerrero
{
    public void Atacar()
    {
        Console.WriteLine("El guerrero ataca con espada");
    }
}

public class Mago
{
    public void Atacar()
    {
        Console.WriteLine("El mago ataca con magia");
    }
}

public class Juego
{
    public static void Main()
    {
        // el cliente crea cada clase concreta a mano
        Guerrero guerrero = new Guerrero();
        guerrero.Atacar();

        Mago mago = new Mago();
        mago.Atacar();

        // agregar un personaje nuevo obliga a modificar Juego
    }
}
```

Fig 6

- Con Factory Method:

```
public interface IPersonaje
{
    void Atacar();
}

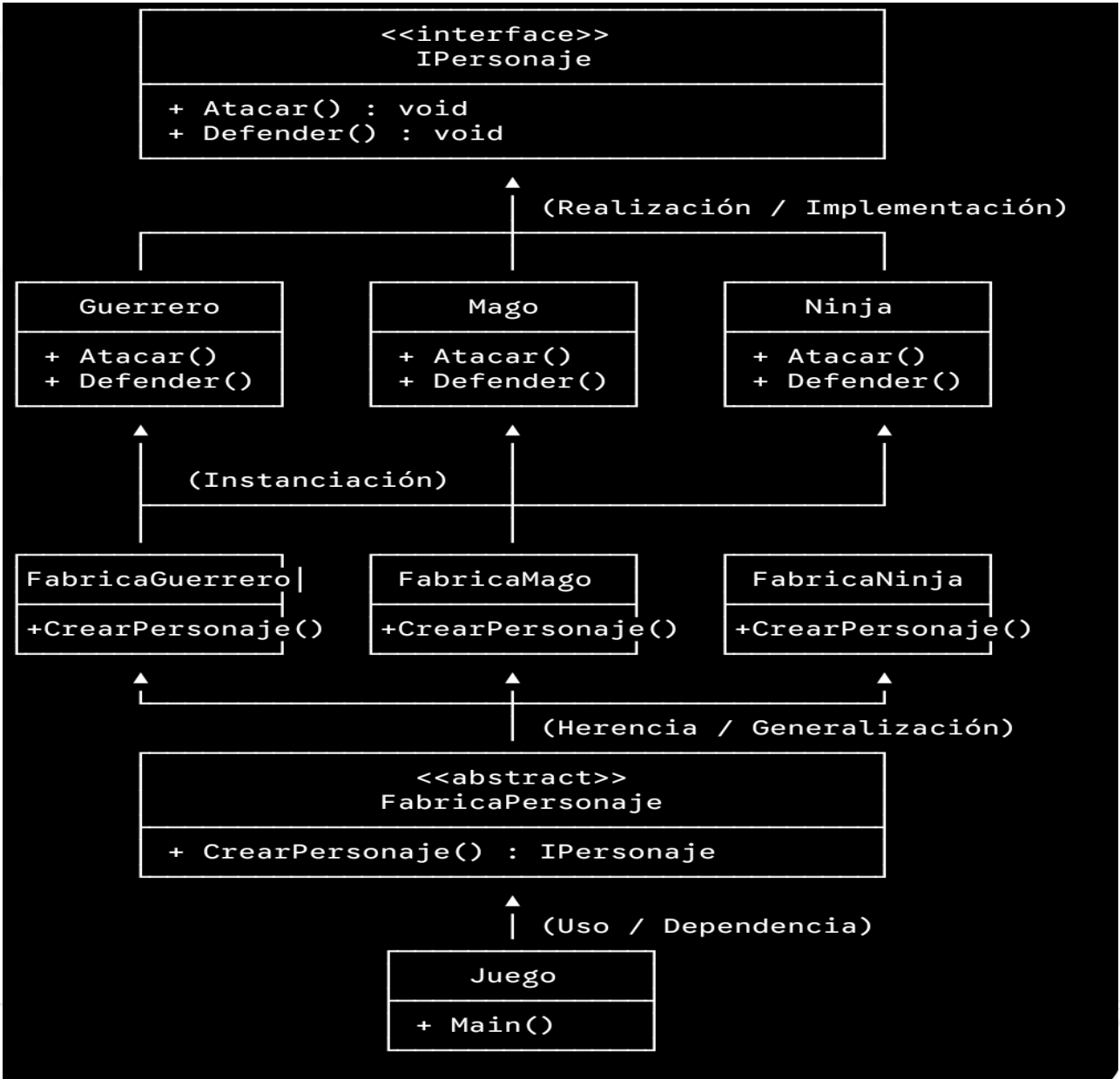
public abstract class PersonajeFactory
{
    public abstract IPersonaje CrearPersonaje();
}

public class GuerreroFactory : PersonajeFactory
{
    public override IPersonaje CrearPersonaje()
    {
        return new Guerrero();
    }
}

public class Juego
{
    public static void Main()
    {
        // el cliente solo conoce la abstracción
        PersonajeFactory factory = new GuerreroFactory();
        IPersonaje personaje = factory.CrearPersonaje();
        personaje.Atacar();

        // un personaje nuevo = una factory nueva, Juego no cambia
    }
}
```

• Con Factory Method: Diagrama



No confundir este patrón con otros conceptos similares:

- **Simple Factory:** Una clase sin herencia ni polimorfismo.
- **Métodos fábrica estáticos:** Si el método se declara static, no hay herencia.
- **Abstract Factory:** Crea familias completas de productos relacionados



Fig 8.

Ventajas y desventajas del patrón.

Ventajas.

- **Desacoplamiento:** Cliente depende de la abstracción, no de clases concretas.
- **Principio de Responsabilidad :** Creación centralizada en un solo punto.
- **Principio Abierto/Cerrado :** Nuevos productos sin modificar código existente.

Desventajas.

- **Mayor complejidad:** Requiere crear varias subclases nuevas.
- **Sobre-ingeniería:** Si solo existe un tipo de producto, el patrón añade clases sin necesidad real.

Referencias

Fig.[1,2,3,4,5] <https://refactoring.guru/design-patterns/factory-method>

Fig.[6,7,8] <https://medium.com/design-bootcamp/the-factory-design-pattern-ba150444c8a7>

Informacion: <https://refactoring.guru/design-patterns/factory-method>

<https://tutorialesprogramacionya.com/disenosoftware/patronesdedisenogofcsharp/tema7.html>



¡Gracias!